Problem Set 6

Due date: Electronic submission of the pdf file of this homework is due on **3/7/2025 before 11:59pm** on canvas.

Name: Chayce Leonard

Resources. Sources

- Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. I primarily consulted the chapters on dynamic programming and amortized analysis methods.
- Kleinberg, J., and Tardos, É. (2005). *Algorithm Design.* Pearson Education. The sections on dynamic programming helped me understand optimal substructure for the coin row problem.
- Demaine, E. "MIT 6.046J Design and Analysis of Algorithms" lecture notes on amortized analysis, which clarified the three methods (aggregate, accounting, and potential) with clear examples.
- Erickson, J. *Algorithms* (available online at http://jeffe.cs.illinois.edu/teaching/algorithms/). His explanations of dynamic programming and amortized analysis helped me verify my solutions.
- Sedgewick, R., and Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. Their sections on dynamic programming implementations provided practical insights for solving recursive problems.

On my honor, as an Aggie, I have neither given nor received any unauthorized aid on any portion of the academic work included in this assignment. This work is my own. Furthermore, I have disclosed all resources (people, books, web sites, etc.) that have been used to prepare this homework. Looking up solutions is prohibited.

Signature: Chayce Leonard _____

Make sure that you describe all solutions in your own words, and that the work presented is your own! Typesetting in LaTeX is required. Read the chapter on amortized analysis in our textbook.

Problem 1 (20 points)**.** Suppose that you have a row of coins with values that are positive integers $v_1, v_2, \ldots, v_n$. These values are not necessarily distinct. The goal is to pick up the highest total value of coins as possible. However, one cannot pick up two coins that are adjacent to each other. (a) Describe an algorithmic solution that is based on the dynamic programming principle. Carefully explain why your solution is correct. (b) What is the solution for coins with values (25,5,5,25,10,10,5,25,1).

Solution. To solve the coin-row problem using dynamic programming, we define a recursive relationship based on the choices available at each coin.

1. Let $dp[i]$ represent the maximum value that can be obtained from the first i coins.
- If there are no coins, the maximum value is 0: $dp[0] = 0$.
- If there is only one coin, the maximum value is the value of that coin: $dp[1] = v[1]$.

3. For each coin i (where i ranges from 2 to n), we have two choices:
- [Include the coin]: If we include the coin $v[i]$, we cannot include the previous coin $v[i-1]$. Thus, the value would be $v[i] + dp[i-2]$.
- [Exclude the coin]: If we exclude the coin $v[i]$, the value would simply be $dp[i-1]$.

Therefore, the recursive relation can be expressed as:

$$dp[i] = \max(dp[i-1], v[i] + dp[i-2])$$

4. [Final Result]: The maximum value obtainable from all n coins will be stored in $dp[n]$.

Now, applying this algorithm to the specific set of coin values:
$(25, 5, 5, 25, 10, 10, 5, 25, 1)$:
- Initialize $v = [0, 25, 5, 5, 25, 10, 10, 5, 25, 1]$ and $n = 9$.
- Initialize $dp = [0] \times (n + 1)$.

Base Cases:
- $dp[0] = 0$
- $dp[1] = 25$

Fill the dp Array:

$$dp[2] = \max(dp[1], v[2] + dp[0]) = \max(25, 5 + 0) = 25$$
$$dp[3] = \max(dp[2], v[3] + dp[1]) = \max(25, 5 + 25) = 30$$
$$dp[4] = \max(dp[3], v[4] + dp[2]) = \max(30, 25 + 25) = 50$$
$$dp[5] = \max(dp[4], v[5] + dp[3]) = \max(50, 10 + 30) = 50$$
$$dp[6] = \max(dp[5], v[6] + dp[4]) = \max(50, 10 + 50) = 60$$
$$dp[7] = \max(dp[6], v[7] + dp[5]) = \max(60, 5 + 50) = 60$$
$$dp[8] = \max(dp[7], v[8] + dp[6]) = \max(60, 25 + 60) = 85$$
$$dp[9] = \max(dp[8], v[9] + dp[7]) = \max(85, 1 + 60) = 85$$

Thus, the maximum value obtainable from the coins is $dp[9] = 85$.

Problem 2 (10 points)**.** In the lecture, we discussed a stack with PUSH, POP, and MULTIPOP operations. If the set of stack operations included a MULTI-PUSH operation, which pushes k items onto the stack, would the $O(1)$ bound on the amortized cost of stack operations continue to hold?

Solution. To determine whether the amortized cost of stack operations remains $O(1)$ when a MULTIPUSH operation is introduced, we analyze the cost of all stack operations, including the new operation, using amortized analysis.
Definitions and Operations The stack operations are defined as follows:
- PUSH: Adds one item to the stack. Cost: $O(1)$.
- POP: Removes one item from the stack. Cost: $O(1)$.
- MULTIPOP: Removes k items from the stack. Cost: $O(k)$.
- MULTIPUSH: Pushes k items onto the stack. Cost: $O(k)$.

Amortized Analysis of the the Multistack structure
Amortized analysis considers the average cost of operations over a sequence of n operations. We will use the aggregate method to analyze the total cost of n operations and divide it by n to find the average cost per operation.

1. Cost of MULTIPUSH
The MULTIPUSH operation pushes k items onto the stack, which takes $O(k)$ time. If MULTIPUSH is used frequently, the total cost of all operations could increase significantly. However, we must consider the total number of items pushed onto the stack over all operations.

2. Aggregate Analysis
Let n be the total number of operations (including PUSH, POP, MULTIPOP, and MULTIPUSH). Over the course of n operations:
- Each item pushed onto the stack (via PUSH or MULTIPUSH) can only be popped once (via POP or MULTIPOP).
- Therefore, the total number of items pushed onto the stack is at most equal to the total number of items popped from the stack.

This means that the total cost of all PUSH and MULTIPUSH operations is proportional to the total number of items pushed, and the total cost of all POP and MULTIPOP operations is proportional to the total number of items popped. Since each item is pushed and popped at most once, the total cost of all operations is $O(n)$.

3. Amortized Cost

The total cost of n operations is $O(n)$. Dividing this by n, the amortized cost per operation is:

$$\text{Amortized cost per operation} = \frac{O(n)}{n} = O(1).$$

4. Solution Conclusion

The introduction of the MULTIPUSH operation does not affect the $O(1)$ amortized cost of stack operations. This is because the total number of items pushed onto the stack (via PUSH or MULTIPUSH) is bounded by the total number of items popped (via POP or MULTIPOP), ensuring that the total cost of all operations remains $O(n)$.

Problem 3 (10 points)**.** In the lecture, we discussed a k-bit counter with an INCREMENT operation. Show that if a DECREMENT operation were included in the k-bit counter example, n operations could cost as much as $\Theta(nk)$ time.

Solution. To demonstrate that including a DECREMENT operation in a k-bit counter could lead to a time complexity of $\Theta(nk)$ for n operations, we need to analyze the worst-case behavior of DECREMENT operations.

For a k-bit binary counter, we know the following:

- The counter stores numbers from 0 to $2^k - 1$
- In the standard INCREMENT operation, the amortized cost is $O(1)$ per operation
- For DECREMENT, we need to analyze its behavior separately

Worst-case for DECREMENT:

Consider decrementing a number that is one less than a power of 2, such as:

$$2^j - 1 = (1\underbrace{11...1}_{j \text{ ones}})_2$$

When we decrement this value, we get:

$$2^j - 2 = (1\quad\underbrace{11...0}_{j-1 \text{ ones, } 1 \text{ zero}}\quad)_2$$

This requires changing only 1 bit, which is an $O(1)$ operation.

However, consider decrementing a power of 2, such as:

$$2^j = (1\underbrace{00...0}_{j \text{ zeros}})_2$$

When we decrement this value, we get:

$$2^j - 1 = (0\underbrace{11...1}_{j \text{ ones}})_2$$

This operation requires changing $j + 1$ bits (from 1 to 0 at position j, and from 0 to 1 for all j positions before it). If $j = k - 1$, then we need to flip all k bits.

Constructing the worst-case sequence:

We can construct a sequence of n operations that achieves the $\Theta(nk)$ bound:

1. Start with the counter at 0
2. Perform INCREMENT until reaching 2^{k-1}
3. Perform DECREMENT to get $2^{k-1} - 1$
4. Perform INCREMENT to get back to 2^{k-1}
5. Repeat steps 3 and 4 as needed

With this sequence, every alternate operation is a DECREMENT of a power of 2, which costs $\Theta(k)$. Since we can make approximately $n/2$ such operations in a sequence of n total operations, the total cost becomes $\Theta(nk)$.

More formally: For n operations, if we arrange them so approximately $n/2$ of them are DECREMENT operations on powers of 2, each costing $\Theta(k)$, then:

Total cost $= \Theta(n/2 \cdot k) + \Theta(n/2 \cdot 1) = \Theta(nk)$

Therefore, with the inclusion of the DECREMENT operation, the cost of n operations on a k-bit counter could be as much as $\Theta(nk)$ time, which is significantly worse than the $\Theta(n)$ time achieved with INCREMENT operations alone.

Problem 4 (20 points)**.** Suppose we perform a sequence of n operations on a data structure in which the i-th operation costs i if i is an exact power of 2, and 1 otherwise. Use aggregate analysis to determine the amortized cost per operation.

Solution. To determine the amortized cost per operation using aggregate analysis, we need to calculate the total cost of n operations and divide by n.

Let's analyze the total cost of n operations:

1. Operations where i is an exact power of 2 (i.e., $i = 2^j$ for some j) cost $i = 2^j$. 2. All other operations cost 1.

The operations that are exact powers of 2 are: 1, 2, 4, 8, 16, 32, ..., $2^{\lfloor \log_2 n \rfloor}$.

Note that $i = 1 = 2^0$ is also a power of 2, so the first operation costs 1.

Total cost of all operations:

$$\begin{aligned}\text{Total cost} &= \sum_{i=1}^{n} \text{cost}(i) \\ &= \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j + \sum_{\substack{i=1 \\ i \neq 2^j \text{ for any } j}}^{n} 1\end{aligned}$$

The first sum represents the cost of operations that are exact powers of 2, and the second sum represents the cost of all other operations.

For the first sum, we have:

$$\sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = 2^{\lfloor \log_2 n \rfloor + 1} - 1 < 2n$$

For the second sum, we have:

$$\sum_{\substack{i=1 \\ i \neq 2^j \text{ for any } j}}^{n} 1 = n - (\lfloor \log_2 n \rfloor + 1) = n - O(\log n)$$

Therefore, the total cost is:

$$\begin{aligned}\text{Total cost} &< 2n + n - O(\log n) \\ &= 3n - O(\log n) \\ &= O(n)\end{aligned}$$

Dividing by n, we get the amortized cost per operation:

$$\text{Amortized cost} = \frac{\text{Total cost}}{n} = \frac{O(n)}{n} = O(1)$$

Therefore, using aggregate analysis, the amortized cost per operation is $O(1)$.

Problem 5 (20 points)**.** Redo Problem 4 using an accounting method of analysis.

Solution. For the accounting method, we assign an amortized cost to each operation and ensure that the sum of amortized costs is greater than or equal to the sum of actual costs. The amortized cost must be sufficient to pay for the current operation and save credits for future expensive operations.

Let's assign an amortized cost of 3 to each operation:

1. For an operation i where i is not a power of 2, the actual cost is 1, so we have 2 extra credits that we can save. 2. For an operation i where $i = 2^j$ for some j, the actual cost is $i = 2^j$.

Now, let's verify that our accounting scheme works:

For any operation $i = 2^j$ (a power of 2), we need to have accumulated enough credits to pay for this expensive operation. The actual cost of this operation is 2^j.

The operations preceding $i = 2^j$ are operations 1 through $2^j - 1$. Among these operations, exactly j operations are powers of 2 (specifically, $2^0, 2^1, ..., 2^{j-1}$), and the rest $(2^j - 1 - j)$ are not powers of 2.

Total credits saved from operations that are not powers of 2:

$$(2^j - 1 - j) \cdot 2 = 2 \cdot 2^j - 2 - 2j$$

Total credits spent on previous power-of-2 operations:

$$\sum_{k=0}^{j-1}(2^k - 3) = \sum_{k=0}^{j-1} 2^k - 3j = 2^j - 1 - 3j$$

Net credits available when we reach operation $i = 2^j$:

$$(2 \cdot 2^j - 2 - 2j) - (2^j - 1 - 3j) = 2^j - 1 + j$$

Since $j > 0$ for $i \geq 1$, we have $2^j - 1 + j > 2^j - 1 \geq 0$. This means we have enough credits to pay for the actual cost of operation $i = 2^j$, which is 2^j.

Since our accounting scheme ensures that we have enough credits to pay for all operations, and the amortized cost per operation is 3, which is $O(1)$, the amortized cost per operation using the accounting method is $O(1)$.

Problem 6 (20 points)**.** Redo Problem 4 using a potential method of analysis.

Solution. For the potential method, we define a potential function $\Phi(D_i)$ that represents the "potential energy" stored in the data structure after the i-th operation. The amortized cost of the i-th operation is calculated as:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Where c_i is the actual cost of the i-th operation.

Let's define the potential function as:

$$\Phi(D_i) = 2i - \sum_{j=0}^{\lfloor \log_2 i \rfloor} 2^j = 2i - (2^{\lfloor \log_2 i \rfloor + 1} - 1) = 2i - 2^{\lfloor \log_2 i \rfloor + 1} + 1$$

Let's verify that this potential function works by analyzing the amortized cost for two cases:

Case 1: i is not a power of 2. The actual cost $c_i = 1$.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + [2i - 2^{\lfloor \log_2 i \rfloor + 1} + 1] - [2(i-1) - 2^{\lfloor \log_2 (i-1) \rfloor + 1} + 1]\end{aligned}$$

Since i is not a power of 2, we know that $\lfloor \log_2 i \rfloor = \lfloor \log_2 (i-1) \rfloor$. Therefore:

$$\begin{aligned}
\hat{c}_i &= 1 + [2i - 2^{\lfloor \log_2 i \rfloor + 1} + 1] - [2(i-1) - 2^{\lfloor \log_2 i \rfloor + 1} + 1] \\
&= 1 + 2i - 2^{\lfloor \log_2 i \rfloor + 1} + 1 - 2i + 2 + 2^{\lfloor \log_2 i \rfloor + 1} - 1 \\
&= 1 + 1 + 2 - 2^{\lfloor \log_2 i \rfloor + 1} + 2^{\lfloor \log_2 i \rfloor + 1} \\
&= 4
\end{aligned}$$

Case 2: i is a power of 2, i.e., $i = 2^k$ for some k. The actual cost $c_i = i = 2^k$.

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\
&= 2^k + [2 \cdot 2^k - 2^{k+1} + 1] - [2(2^k - 1) - 2^{\lfloor \log_2 (2^k - 1) \rfloor + 1} + 1]
\end{aligned}$$

Since $2^k - 1$ is not a power of 2, $\lfloor \log_2 (2^k - 1) \rfloor = k - 1$. Therefore:

$$\begin{aligned}
\hat{c}_i &= 2^k + [2 \cdot 2^k - 2^{k+1} + 1] - [2(2^k - 1) - 2^{(k-1)+1} + 1] \\
&= 2^k + [2 \cdot 2^k - 2^{k+1} + 1] - [2 \cdot 2^k - 2 - 2^k + 1] \\
&= 2^k + 2 \cdot 2^k - 2^{k+1} + 1 - 2 \cdot 2^k + 2 + 2^k - 1 \\
&= 2 \cdot 2^k - 2^{k+1} + 2 \\
&= 2 \cdot 2^k - 2 \cdot 2^k + 2 \\
&= 2
\end{aligned}$$

Therefore, the amortized cost using the potential method is $O(1)$ for both cases.

Additionally, we need to verify that the potential function is initially zero and never becomes negative. Since $\Phi(D_0) = 0$ and $\Phi(D_i) > 0$ for all $i > 0$ (as can be verified by checking small values of i and noting that $\Phi(D_i)$ increases as i increases), our potential function satisfies these conditions.

In conclusion, using the potential method, the amortized cost per operation is $O(1)$.

Discussions on canvas are always encouraged, especially to clarify concepts that were introduced in the lecture. However, discussions of homework problems on canvas should not contain spoilers. It is okay to ask for clarifications concerning homework questions if needed. Make sure that you write the solutions in your own words.

Checklist:

- ☐ Did you add your name?
- ☐ Did you disclose all resources that you have used?
  (This includes all people, books, websites, etc. that you have consulted)
- ☐ Did you sign that you followed the Aggie honor code?
- ☐ Did you solve all problems?
- ☐ Did you submit the pdf file of your homework?